

Time and Space Complexity

25 February 2026 19:57

Two criteria are used to judge algorithms:

1. Time complexity
2. Space complexity

Time complexity of an algorithm is the amount of CPU time it needs to run completion.

Space complexity of an algorithm is the amount of memory it needs to run completion.

TIME:

- Operations
- Comparisons
- Loops
- References
- Function calls to outside

SPACE:

- Variables
- Data Structures(Array, LinkedList Tree, etc.)
- Allocations
- Function call (Memory STACKs)

What is time complexity?

Time complexity of an algorithm is the amount of time(or number of steps) needed by a program to complete its task(to execute a particular algorithm)

The time taken for an algorithm is comprised of two times:

1. Compilation time
2. Run time

Compile time:

- Compilation time is the time taken to compile an algorithm
- While compiling it checks the syntax and semantic errors in the program and it links it with the standard libraries.

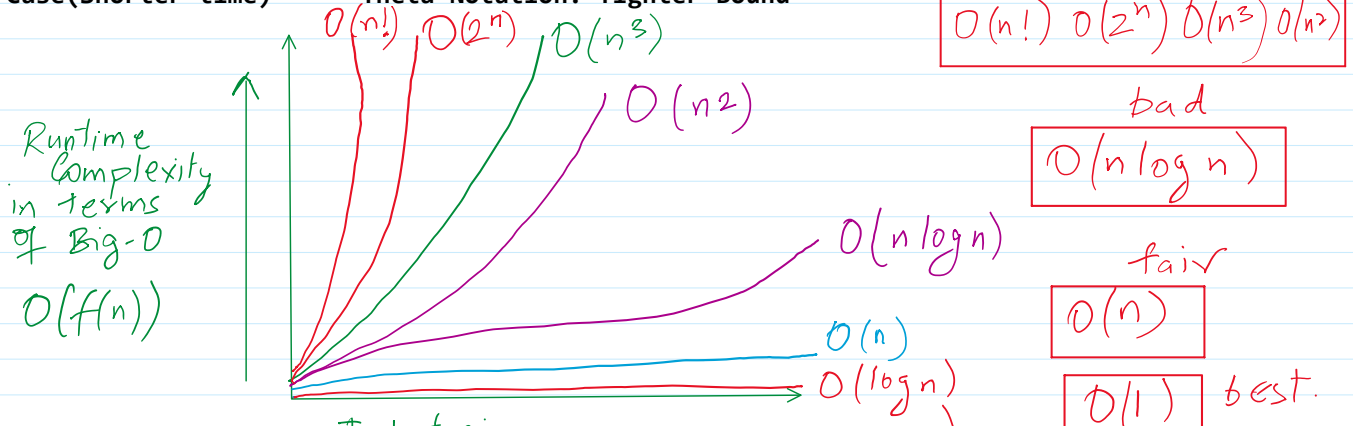
Run time:

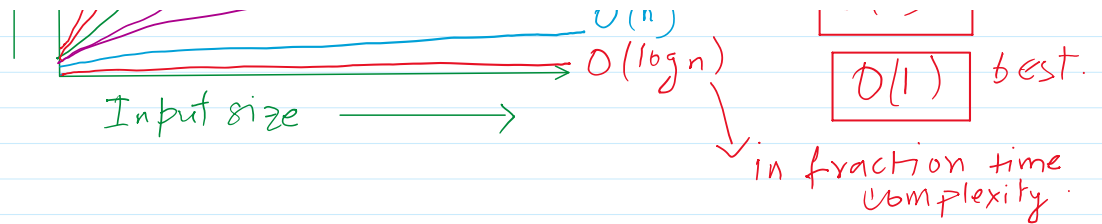
- It is a time to execute the compiled program
- The runtime of an algorithm depends on the number of instructions present in the algorithm
- Note that the runtime is calculated only for executable statements and not for declaration statements

Types of time complexity

1. Worst Case(longest time)
2. Average Case(Average time)
3. Best Case(Shorter time)

Big Oh Notation: Upper Bound
Omega Notation: Lower Bound
Theta Notation: Tighter Bound





```

boolean isPrime(int x) {
    int i, count = 0;
    for (i = 1; i <= x; i++) {
        if (x % i == 0) {
            count++;
        }
    }
    if (count == 2) {
        return true;
    } else {
        return false;
    }
}

```

Handwritten calculations for the primality of 11357962:

$$\begin{array}{r}
 11357962 \\
 \times 3 \\
 \hline
 33969
 \end{array}$$

$$\begin{array}{r}
 11357962 \\
 \times 63 \\
 \hline
 715540506
 \end{array}$$

$$\begin{array}{r}
 11357962 \\
 \times 666 \\
 \hline
 7544302572
 \end{array}$$

$$\begin{array}{r}
 11357962 \\
 \times 6729 \\
 \hline
 76400000000
 \end{array}$$

$$\begin{array}{r}
 11357962 \\
 \times 235 \\
 \hline
 266811070
 \end{array}$$

$$\begin{array}{r}
 11357962 \\
 \times 189 \\
 \hline
 2146654818
 \end{array}$$

$$\begin{array}{r}
 11357962 \\
 \times 4619 \\
 \hline
 52465671358
 \end{array}$$

$$\begin{array}{r}
 11357962 \\
 \times 3996 \\
 \hline
 45388302352
 \end{array}$$

$$\begin{array}{r}
 11357962 \\
 \times 62361 \\
 \hline
 708000000000
 \end{array}$$

$x = 11357962$ $O(x)$

The number which is divisible by 1 and itself only, then it is a prime number. For example, (2) 3, 5, 7, 11, 13, 17, 19, 23, 29.

```

boolean isPrime(int x) {
    if (x > 2 && x % 2 == 0)
        return false;
    for (int od = 3; od <= Math.sqrt(x); od += 2) {
        if (x % od == 0)
            return false;
    }
    return true;
}

```

Annotations: $\sqrt{x}$ is circled in the loop condition. A box contains the values 3, 5, 7, 9. A note 'worst case complexity' points to the loop. The time complexity is given as $O(\frac{\sqrt{n}}{2})$ and $(10)/2$ Time complexity.